Automated Analysis of Flakiness-mitigating Delays

Jean Malm
Adnan Causevic
Björn Lisper
{jean.malm,adnan.causevic,bjorn.lisper}@mdh.se
Mälardalen University
Västerås, Sweden

Sigrid Eldh
sigrid.eldh@ericsson.com
Ericsson AB
Stockholm, Sweden

ABSTRACT

During testing of parallel systems, which allow asynchronous communication, test flakiness is sometimes avoided by explicitly inserting delays in test code. The choice of delay approach can be a trade-off between short-term gain and long-term robustness. In this work, we present an approach for automatic detection and classification of delay insertions, with the goal of identifying those that could be made more robust. The approach has been implemented using an open-source compiler tooling framework and validated using test code from the telecom industry.

CCS CONCEPTS

• **Software and its engineering → Software testing and debugging**; **Maintaining software**; Software creation and management; • **Theory of computation** → Program analysis.

KEYWORDS

Static analysis, Test smells, Test code metrics, flakiness, Technical debt

ACM Reference Format:
Jean Malm, Adnan Causevic, Björn Lisper, and Sigrid Eldh. 2020. Automated Analysis of Flakiness-mitigating Delays. In *IEEE/ACM 1st International Conference on Automation of Software Test (AST '20), October 7–8, 2020, Seoul, Republic of Korea.* ACM, New York, NY, USA, 4 pages. https://doi.org/10.1145/3387903.3389320

1 INTRODUCTION

The ever-growing use of software systems in our daily lives makes quality assurance a vital part of the development process. Given the need for agility to stay competitive, the continuous integration (CI) development model is used more and more in industry. One feature in CI toolchains is the ability to automatically run a set of tests when code is being submitted for merging into production. These can also be set to be "gating", in the sense that if they fail, they will not allow integration into production code [6]. This quality control model depends on test verdicts being trustworthy. Unfortunately, this is not always the case. Factors such as systems not being designed with testing in mind or by environmental interactions not taken into account during the test design [4] can lead to erratic test verdicts. Such tests are commonly called *flaky* and can be described as a test that when applied to a system S yields different outcomes on different occasions in apparently identical test scenarios.

Several causes of flakiness are timing-related. One cause is due to an unintended execution ordering between test and production code. It occurs when the test tries to assert something before the result is ready, and is found where the communication between test and test software is done asynchronously [6][7].

To avoid this type of flakiness, Luo et al. noted developers explicitly enforcing the correct execution order using delays in test code fixes. This was done by inserting a delay before the assertion, either by a fixed duration (sleep) or by using a polling/event-based approach (waitFor). Sleeps do not necessarily remove the underlying issue [7], as these times are quite environment- and load-dependent. To remedy this, it is possible to use an additional safety margin on delay times to be robust, which has a negative impact on test runtimes. However, the low up-front implementation cost may lead to developers choosing this alternative in order to keep up productivity or rapidly mitigate flakiness before deadlines or before a better fix is available [1].

In this work, we present a method for performing automated context-dependent classification of delays to find sleep-based delays. In addition, we gather delay durations as an additional test effectiveness metric. The method has been implemented as a clang checker and has been tested on an industrial codebase for baseband software in the telecom domain. We aim to answer the following research questions:

- RQ1 What does the distribution and type of delays look like in a sample of a large-scale distributed industrial system?
- RQ2 What are some common usages of delays in test code for a certain sample of the industrial system?

In the process, we make the following contributions: 1) A method for aiding the identification and classification of potentially unstable delays in tests. 2) Observations of delay usage in test suites for concurrent systems.

2 BACKGROUND

Several studies on test flakiness, both in the open-source [9] [7] and industrial [6] contexts, have focused on identification of causes and mitigation strategies.

In [7], the authors created a taxonomy of flakiness issues. They noted that the top three causes found and fixed by developers within the study were timing-related: async-wait, concurrency issues, and test execution ordering being the third. To reduce async-waits, e.g.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
AST '20, October 7–8, 2020, Seoul, Republic of Korea
© 2020 Association for Computing Machinery.
ACM ISBN 978-1-4503-7957-1/20/05…$15.00
https://doi.org/10.1145/3387903.3389320

enforce the correct execution sequence, they noted that developers typically added a delay to the test. Delays can be separated into two types: *sleep* or *wait-for*.

Wait-for guarantees that the test assertion will not be done until some event happens (either the requested data is available and ready, or some timeout occurs due to waiting too long). However, it requires the ability to observe the specific information in the SUT in order to determine when it's safe to continue. This may not be possible in all cases, for example due to the design and complexity of the system [4] or inability to rapidly affect the operation of certain components, such as the case when existing hardware is reused.

Sleeps are comparatively cheap to implement, assuming existing support for forcing context switches and task suspension. The difficulty is finding a duration that are both safe and efficient. Short delays may not give enough time to prevent flakiness, whereas long ones reduce test execution effectiveness. It is therefore a trade-off between having a time that is long enough to make the testing process suitably robust, while still being as small as possible. Durations are affected by many different factors such as execution time of the SUT, but also on environmental factors such as CPU- or network load. In addition, it can also be affected by the type of environment that the test is being run in, for example in the case of using hardware-in-the-loop simulation.

2.1 Program Analysis

To complement testing, different types of program analysis processes can be used to automatically gather data about sought properties of a program. Such techniques are separated into two complementary families: static and dynamic analysis, and are used to some success in large-scale software systems [4].

Dynamic program analysis techniques monitor the program during runtime in order to find run-time issues, such as those related to performance or memory usage. The benefit is that it analyses an actual execution of a program, meaning that you get accurate data, but this also means that it requires compilation and deployment of the software, and is dependent on the quality and coverage of the test inputs.

Static analysis works on some abstract version of the program, e.g. graphs or code. It can be used during compilation to optimise and warn about potential syntactic and semantic issues in the code, thereby enforcing design rules through compilation warnings or analysis verdicts. In addition, there are analyses such as symbolic execution [2] that attempt to explore all possible executions paths of the code. The benefits is that static analysis can be done early, without having to compile and deploy all parts of the program, but one downside is that it can give a high number of false positives, which can cause developers to ignore potential issues [5].

3 METHOD

In this section, we describe the heuristic method used to analyse and extract data from delays found in the code. It consists of a number of steps, which can be seen in figure 1. The method takes as input the code to analyse, as well as the delay functions to target for the analysis. For the classification, an initial mapping from a function to $type \in \{Sleep, WaitFor\}$ is required.

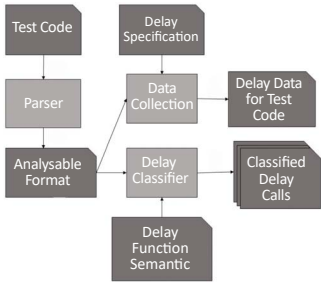


Figure 1: The method used to perform the classification and data extraction on delays in test code.

For duration extraction, some specification-based data should be provided. This tells the analysis how to extract the duration (e.g. which argument to check), and how to convert it to some common time unit. Here, we represent it as a mapping from function IDs to a tuple $(ind, convRate)$ where ind is the index of the argument holding the delay duration and $convRate$ is the conversion rate to use for the particular function.

To perform the analysis, the code is parsed into some appropriate representation, which may differ based on individual needs. If the classification of delays can be done solely based on code patterns, the *abstract syntax tree* (AST) representation is enough. If it requires some flow information, the *control-flow graph* (CFG) is better.

Durations however may be dependent on specific data flows. For instance, if the values used are not constant, some data value analysis is necessary. Additionally there can be cases where a delay is executed multiple times, and therefore the potential number of executions should be bounded. As these analyses are typically done on the CFG, duration extraction targets the CFG representation.

The data extraction process uses the information given to evaluate and extract relevant arguments, and then process them accordingly. This might include combining the evaluated duration with information about the program flow. One example is the number of delay executions, which may be of interest for investigating timeout durations of polling loops.

The classification process works as follows: If the function found has been tagged as a waitFor, this is the result of the classifier. On the other hand, if it is a sleep-type, we additionally need to check if it is inside a polling loop, which should be classified as waitFor, as stated in [7].

WaitFor-based polling loops can have a number of appearances, but is in general a loop containing a delay to control the polling frequency, and looping stops based on one of two conditions. Either the polled data reaches some expected value, or it triggers some timeout watchdog to avoid polling forever. One sign that the loop is a polling one is if there is an if-statement containing a break, as this would terminate the loop early.

4 IMPLEMENTATION

As a proof of concept, the analysis method described in Section 3 was implemented as DelayChecker, a clang[1] checker. Clang is a C-type language frontend to the LLVM compiler framework. It

[1]https://clang.llvm.org/

provides both a compiler driver as well as support for creating checkers as modular code analysis entities.

DelayChecker uses the AST for classification. Clang provides a domain-specific language for creating pattern matching expressions for AST trees. These can then be registered with callback functions, firing when the analyser finds a match. To implement the checks, we wrote such expressions for each delay type, based on commonly observed cases in the use case.

The static analysis engine was used to extract both data- and program-flow information. As it is based on symbolic execution, it attempts to execute all possible paths in the program, which includes forcing loops to execute as many times as possible. This gives us the possibility to bound the number of individual delay executions, even inside loops. This in turn gives us an upper bound on the number of context switches and the delay duration for a given test, which was noted by stakeholders to be of interest from a test effectiveness perspective.

It should be noted that the usage of symbolic execution for this purpose was done to enable data collection in the target environment. In general it does not scale well for large programs, due to path explosion, but for this particular system the tests do not have a lot of branches, and the number of loop iterations are relatively low, resulting in less paths to explore.

5 VALIDATION PROCESS

The method was validated using the prototype tool described in section 4. It was run by including it in the compilation process of a number of test suites for testing radio communication software.

This system has a high demand on the level of concurrent processing, and is therefore developed in accordance to the actor model. There, the system is composed of a number of inter-operating tasks that communicate mainly by message passing. Depending on the content of the message, actors will take one of several actions, such as spawning new actors specifically to handle the requests [8]. Given that the system has some real-time requirements, there are cases where actors will have to communicate results back by writing directly to memory, which in the case of tests will result in a lack of observability until the value is asserted.

5.1 Research Question 1

As noted in the method section, we require knowledge of which functions are used to implement/insert delays into the tests. To gather this information, we searched through both the codebase and commit logs looking for delay-related keywords (e.g. "wait", "delay", "sleep"), which was followed up by reading some documentation. A number of sleep and wait-type delay functions were extracted and classified.

To validate the classifier, we targeted specifically test suites that had pre-existing delays, found through a context-independent search. The existing build system allowed for specific checkers to run on files specified by name patterns, which was used to focus specifically on files containing test cases.

In this way, we could use DelayChecker to find and classify the delay functions based on their context, using the proposed method. The resulting distribution can be seen in Figure 2, which shows the number of found sleep-type function calls compared to the number

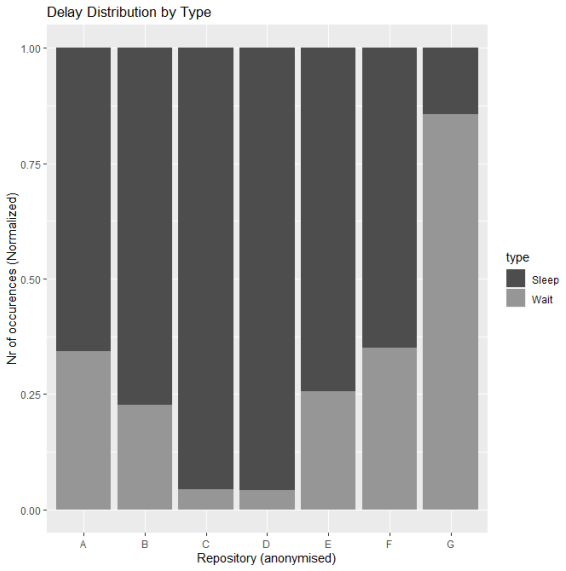


Figure 2: Classification of delays in repositories. Please note that for the purpose of anonymisation, the number of occurrences are presented as percentages. The amount of sleeps for the given sub-repos were (A,B,C,D,E,F,G) = (65.561%, 77.293%, 95.640%, 95.833%, 74.419%, 65.021%, 14.286%)

of wait-types. Overall, the number of sleeps ended up representing 70.3399245% of the total amount of delays found in this slice of the system. What is noteworthy is that most of these sleeps could be found in tests for testing legacy functionality, whereas the newer systems are designed more towards observability, which cuts down on the needs for sleeps.

Additionally. we found that the AST-based classifier handles cases well when the delay is not in a loop, or when the polling is stopped using an if-then-break. However, the classifier exhibits some false classifications for some special cases of polling structures not considered in the initial implementation.

As part of the study, we also used the static analysis engine to retrieve durations and execution bounds on delays. These are typically in the range of milliseconds to a few seconds for sleeps, depending on the testing level. When polling was supported, individual polling times were typically expressed in microseconds, with a maximum loop bound in the [100, 1000] range. So for an individual test, these delays are not costing much, but as tests are re-run frequently, this adds up over time, especially on an organisational level. To check the potential savings, we reduced the durations in some test cases until flakiness was reintroduced. For some suites, we could decrease them by 50% before any tests started failing reliably.

5.2 Research Question 2

Based on the classification, we noted a high number of sleep-type delays in the test code. Upon manually inspecting the results we saw a few common usages:

5.2.1 Enforced Execution Ordering. The most common usage of delays in test code for this system is to enforce execution orderings. This aligns with existing observations by e.g. [7] that delays are used to reduce timing-related flakiness. Due to the high amount of concurrency in the system, many sleeps are inserted to force a context switch, thereby giving concurrently executing actions enough time to finish. From reading the related code comments, this was initially done as the system was not designed to notify tests of the readiness of data. This could be a motivating factor to use such a solution, at least initially, before a redesign is done and put into production.

5.2.2 Reduce unnecessary CPU usage. Another use of sleep-delay is to reduce the number of pollings. One example was that after sending some signal, tests would first sleep for a bit, and only then start polling for the response. Given the comparatively high polling rate (typically two orders of magnitude), this reduces the number of polls significantly than had they started polling immediately. This however assumes that testers know for a fact that the result cannot possibly be available before some minimum duration, which may require some manual work to verify.

6 CONCLUSION

In this work, we have studied the usage of delays in test code. To check for potential robustness issues in the system, we developed a context-dependent classification for delays. This was applied on a subset of an industrial system in order to find potential improvement possibilities w.r.t. the types of delays used.

The results seem to indicate that for the repositories chosen, most of the delays inserted have historically been sleep-types. Sleeps on their own are not considered a reliable approach in academia, due to its brittleness w.r.t. changes in the system or environment [7]. But in situations where there is a need for rapid changes and fixes, they can be used as an initial step to lower the non-determinism in testing.

6.1 Related Work

The issues caused by non-deterministic tests have gained some attention in research over the past years. In [7], the authors present a taxonomy for flakiness causes, and suggest ways to fix them. They found adding delays to be the common way to fix flakiness they classify as "async-wait", and that delays could be either sleep or waitFor types, which we are using in the classifier in this work.

In [6], Lam et al. performed a root cause analysis of flakiness within Microsoft, noting that even a small amount of flakiness may result in a significant fixing effort. They analysed execution logs to identify flaky causes, while our approach focuses on using static analysis for early detection of potential flakiness related to usage of sleep-type delays.

Tools exist for finding sleep-related test smells, for instance *SonarSource*[2] or the research prototype mentioned in [9]. However these tools target the Java language, whereas we target an industrial context where an in-house flavour of C is used, and has a number of delay-type functions depending on the subsystem targeted. In addition, our process provides a way to get additional delay metrics based on symbolic execution.

Greiler et al. [3] used extraction of certain metrics in order to find a number of *test fixture* smells by comparing these with threshold values. This approach could be applied to our method as well, in order to improve quality. For instance, it can be used to find sleep durations that are deemed too high by developers.

6.2 Future work

To further validate the method, we intend to apply it on a more extensive set of test code, both within industry but also open-source e.g. those proposed in [9]. The accuracy of the classifier could be improved upon w.r.t. classification of delays inside polling loops.

Given the amount of sleep-type delays in the studied tests, some automated approach for refactoring sleeps into polling would improve the robustness of the tests themselves. Given that polling may cause a number of context switches and in general waste CPU time, refactoring them into event- or interrupt-based waits would improve efficiency. Another aspect of this would be to use existing delay patterns in order to automatically derive new insertion points for delays, to help developers.

ACKNOWLEDGMENTS

This work is supported by Vinnova, Sweden's innovation agency through the ITEA3 TESTOMAT Project (2017-02269)[3].

The authors would like to thank the contacts at the company for their time and help with access and the tool validation process.

REFERENCES

[1] Gabriele Bavota, Abdallah Qusef, Rocco Oliveto, Andrea De Lucia, and Dave Binkley. 2015. Are test smells really harmful? An empirical study. *Empirical Software Engineering* 20, 4 (01 Aug 2015), 1052–1094. https://doi.org/10.1007/s10664-014-9313-0

[2] Cristian Cadar and Koushik Sen. 2013. Symbolic Execution for Software Testing: Three Decades Later. *Commun. ACM* 56, 2 (Feb. 2013), 82–90. https://doi.org/10.1145/2408776.2408795

[3] M. Greiler, A. van Deursen, and M. Storey. 2013. Automated Detection of Test Fixture Strategies and Smells. In *2013 IEEE Sixth International Conference on Software Testing, Verification and Validation*. 322–331. https://doi.org/10.1109/ICST.2013.45

[4] M. Harman and P. O'Hearn. 2018. From Start-ups to Scale-ups: Opportunities and Open Problems for Static and Dynamic Program Analysis. In *2018 IEEE 18th International Working Conference on Source Code Analysis and Manipulation (SCAM)*. 1–23. https://doi.org/10.1109/SCAM.2018.00009

[5] Brittany Johnson, Yoonki Song, Emerson Murphy-Hill, and Robert Bowdidge. 2013. Why Don't Software Developers Use Static Analysis Tools to Find Bugs?. In *Proceedings of the 2013 International Conference on Software Engineering (ICSE '13)*. IEEE Press, 672–681.

[6] Wing Lam, Patrice Godefroid, Suman Nath, Anirudh Santhiar, and Suresh Thummalapenta. 2019. Root Causing Flaky Tests in a Large-scale Industrial Setting. In *Proceedings of the 28th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2019)*. ACM, New York, NY, USA, 101–111. https://doi.org/10.1145/3293882.3330570

[7] Qingzhou Luo, Farah Hariri, Lamyaa Eloussi, and Darko Marinov. 2014. An Empirical Analysis of Flaky Tests. In *Proceedings of the 22Nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE 2014)*. ACM, New York, NY, USA, 643–653. https://doi.org/10.1145/2635868.2635920

[8] Abu Naser Masud, Björn Lisper, and Federico Ciccozzi. 2018. Automatic Inference of Task Parallelism in Task-graph-based Actor Models. *Journal of IEEE Access* 99 (December 2018), 1–27. http://www.es.mdh.se/publications/5319-

[9] Fabio Palomba and Andy Zaidman. 2019. The smell of fear: on the relation between test smells and flaky tests. *Empirical Software Engineering* 24, 5 (01 Oct 2019), 2907–2946. https://doi.org/10.1007/s10664-019-09683-z

[2] https://www.sonarsource.com/

[3] https://www.testomatproject.eu/